

1) Write the Python program to solve the 8-Puzzle Problem.

Aim

To write a Python program to solve the 8-Puzzle problem using Breadth First Search (BFS).

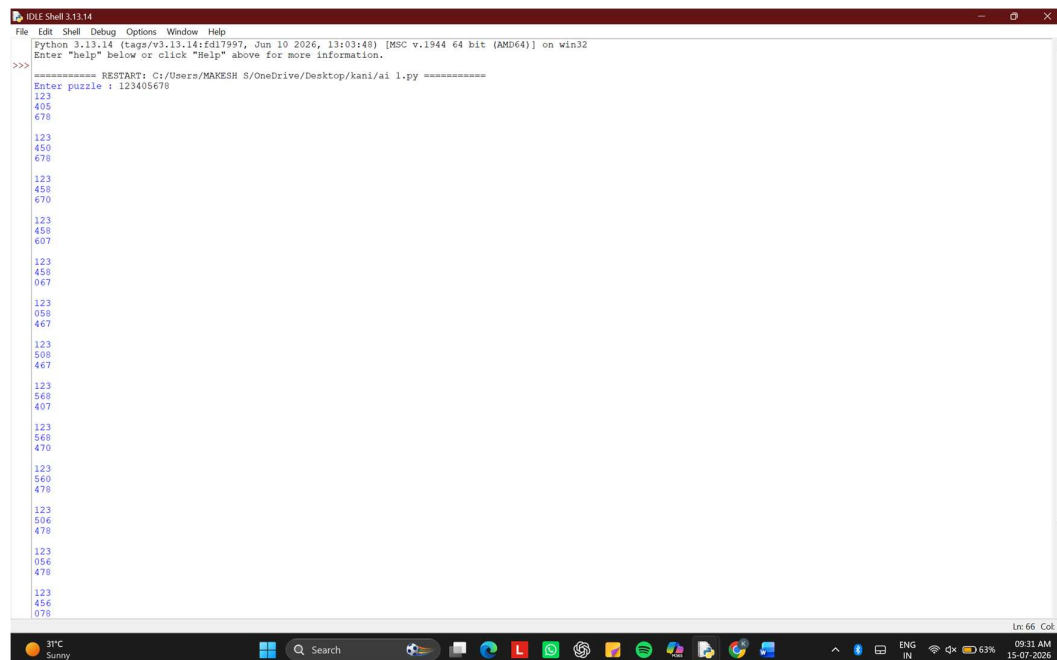
Algorithm

1. Read the initial puzzle state.
2. Define the goal state.
3. Find the position of the blank tile (0).
4. Generate all possible moves.
5. Use BFS to explore states.
6. Stop when the goal state is reached.
7. Display the solution path.

Code

```
from collections import deque
goal = "123456780"
moves = {
    0: [1,3], 1: [0,2,4], 2: [1,5],
    3: [0,4,6], 4: [1,3,5,7], 5: [2,4,8],
    6: [3,7], 7: [4,6,8], 8: [5,7]
}
def bfs(start):
    q = deque([(start, [])])
    visited = set()
    while q:
        state, path = q.popleft()
        if state == goal:
            return path + [state]
        if state in visited:
            continue
        visited.add(state)
```

Output



2) Write the Python program for the Water Jug Problem.

Aim

To write a Python program to solve the Water Jug problem.

Algorithm

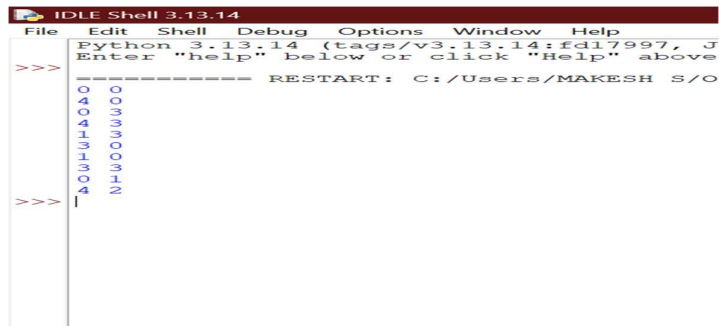
1. Initialize both jugs as empty.
2. Fill or empty the jugs as required.
3. Pour water from one jug to another.
4. Repeat the process until the target quantity is obtained.
5. Display each step.

Code

```
from collections import deque

def water_jug(x, y, target):
    q = deque([(0, 0)])
    visited = set()
    while q:
        a, b = q.popleft()
        if (a, b) in visited:
            continue
        visited.add((a, b))
        print(a, b)
    if a == target or b == target:
        return
    q.extend([
        (x, b), (a, y),
        (0, b), (a, 0),
        (max(0, a-(y-b)), min(y, a+b)),
        (min(x, a+b), max(0, b-(x-a)))
    ])

water_jug(4, 3, 2)
```



```
IDLE Shell 3.13.14
File Edit Shell Debug Options Window Help
Python 3.13.14 (tags/v3.13.14:fd17997, J
Enter "help" below or click "Help" above
>>>
40340
>>> |
RESTART: C:/Users/MAKESH S/O
```

Result

Thus, the Python program for the Water Jug problem was executed successfully and the target quantity of water was obtained.

3. Write the Python program for the Crypt-Arithmetic Problem.

Aim

To write a Python program to solve the Crypt-Arithmetic problem.

Algorithm

1. Read the given words.
2. Assign unique digits to each letter.
3. Ensure leading letters are not zero.
4. Check whether the arithmetic equation is satisfied.
5. Display the valid digit assignment.

Code

```
from itertools import permutations
letters = ('S','E','N','D','M','O','R','Y')
for p in permutations(range(10), 8):
    d = dict(zip(letters, p))
    if d['S']==0 or d['M']==0:
        continue
```

```

send = 1000*d['S']+100*d['E']+10*d['N']+d['D']
more = 1000*d['M']+100*d['O']+10*d['R']+d['E']
money = 10000*d['M']+1000*d['O']+100*d['N']+10*d['E']+d['Y']

if send + more == money:

    print(d)

    print(send, "+", more, "=", money)

    break

```

```

Python 3.13.14 (tags/v3.13.14:f017997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/MAKESH S/OneDrive/Desktop/kani/ai 3.py =====
{'S': 9, 'E': 5, 'N': 6, 'D': 7, 'M': 1, 'O': 0, 'R': 8, 'Y': 2}
9567 + 1085 = 10652
>>>

```

Result

Thus, the Python program for the Crypt-Arithmetic problem was executed successfully and the correct digit assignment was obtained.

4. Write the Python program for the Missionaries and Cannibals Problem.

Aim

To write a Python program to solve the Missionaries and Cannibals problem.

Algorithm

1. Define the initial and goal states.
2. Generate all valid boat moves.
3. Check that missionaries are never outnumbered by cannibals.
4. Use BFS to explore valid states.
5. Continue until the goal state is reached.
6. Display the solution path.

Code

```

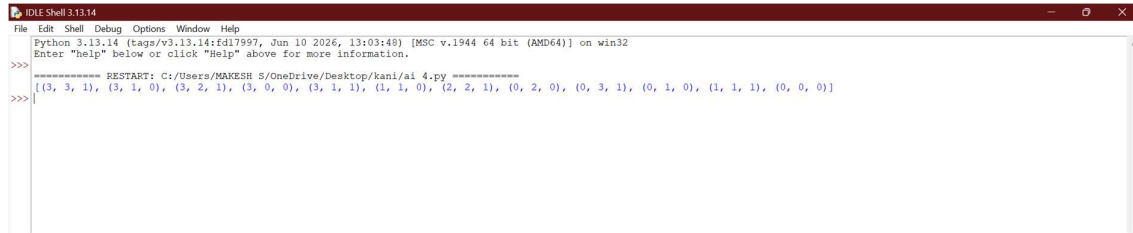
from collections import deque

```

```

start = (3,3,1)
goal = (0,0,0)
moves = [(1,0),(2,0),(0,1),(0,2),(1,1)]
def valid(m,c):
    if m<0 or c<0 or m>3 or c>3:
        return False
    if m and m<c:
        return False
    if 3-m and 3-m < 3-c:
        return False
    return True
q = deque([(start, [])])
visited = set()
while q:
    state, path = q.popleft()
    if state == goal:
        print(path+[state])
        break
    if state in visited:
        continue
    visited.add(state)
    m,c,b = state
    for dm,dc in moves:
        if b:
            nm,nc = m-dm,c-dc
        else:
            nm,nc = m+dm,c+dc
        if valid(nm,nc):
            q.append(((nm,nc,1-b), path+[state]))

```



```
IDE Shell 3.13.14
File Edit Shell Debug Options Window Help
Python 3.13.14 (tags/v3.13.14:f417997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
RESTART: C:/Users/MAKESH S/OneDrive/Desktop/kani/ai 4.py
>>> [(3, 3, 1), (3, 1, 0), (3, 2, 1), (3, 0, 0), (3, 1, 1), (1, 1, 0), (2, 2, 1), (0, 2, 0), (0, 3, 1), (0, 1, 0), (1, 1, 1), (0, 0, 0)]
```

Result

Thus, the Python program for the Missionaries and Cannibals problem was executed successfully and the solution path was obtained.

5. Write the Python program for the Vacuum Cleaner Problem.

Aim

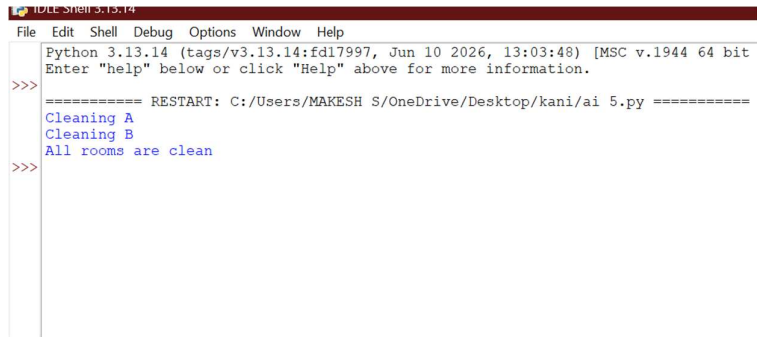
To write a Python program to simulate the Vacuum Cleaner problem.

Algorithm

1. Initialize the environment with dirty rooms.
2. Place the vacuum cleaner in the starting room.
3. Check the current room.
4. If the room is dirty, clean it.
5. Move to the next room.
6. Repeat until all rooms are clean.
7. Display the final status.

Code

```
rooms = {'A':'Dirty','B':'Dirty'}
location = 'A'
while 'Dirty' in rooms.values():
    if rooms[location] == 'Dirty':
        print("Cleaning", location)
        rooms[location] = 'Clean'
    location = 'B' if location == 'A' else 'A'
print("All rooms are clean")
```



```
IDLE Shell 3.13.14
File Edit Shell Debug Options Window Help
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit]
Enter "help" below or click "Help" above for more information.
>>>
==== RESTART: C:/Users/MAKESE S/OneDrive/Desktop/kani/ai 5.py =====
Cleaning A
Cleaning B
All rooms are clean
>>>
```

Result

Thus, the Python program for the Vacuum Cleaner problem was executed successfully and all the rooms were cleaned.

6. Write the Python program to implement BFS (Breadth First Search).

Aim

To write a Python program to implement Breadth First Search (BFS) traversal of a graph.

Algorithm

1. Create a graph using an adjacency list.
2. Initialize a queue and a visited list.
3. Insert the starting node into the queue.
4. Remove the front node from the queue.
5. Visit all unvisited adjacent nodes and add them to the queue.
6. Repeat until the queue becomes empty.
7. Display the traversal order.

Program

```
graph = {
    'A':['B','C'],
    'B':['D','E'],
    'C':['F'],
    'D':[],
```

```

'E':['F'],
'F':[]
}
visited = []
queue = ['A']
while queue:
    node = queue.pop(0)
    if node not in visited:
        print(node, end=" ")
        visited.append(node)
        queue.extend(graph[node])

```

Output

```

>>>
===== REST?
A B C D E F
>>>

```

Result

Thus, the Python program to implement Breadth First Search (BFS) was executed successfully.

7. Write the Python program to implement DFS (Depth First Search).

Aim

To write a Python program to implement Depth First Search (DFS) traversal of a graph.

Algorithm

1. Create a graph.
2. Mark all nodes as unvisited.
3. Visit the starting node.
4. Recursively visit all adjacent unvisited nodes.
5. Continue until all nodes are visited.
6. Display the traversal order.

Program

```

graph = {

```

```

'A':['B','C'],
'B':['D','E'],
'C':['F'],
'D':[],
'E':['F'],
'F':[]
}
visited = set()
def dfs(node):
    if node not in visited:
        print(node, end=" ")
        visited.add(node)
        for i in graph[node]:
            dfs(i)
dfs('A')

```

Output

```

>>> ===== REST?
A B C D E F
>>> |

```

Result

Thus, the Python program to implement Depth First Search (DFS) was executed successfully.

8. Write the Python program to implement Travelling Salesman Problem (TSP).

Aim

To write a Python program to find the minimum travelling cost using the Travelling Salesman Problem.

Algorithm

1. Read the cost matrix.
2. Generate all possible paths.
3. Calculate the total cost of each path.
4. Find the minimum cost.

5. Display the minimum travelling cost.

Program

```
from itertools import permutations

graph = [
    [0,10,15,20],
    [10,0,35,25],
    [15,35,0,30],
    [20,25,30,0]
]

n = 4

cities = range(1, n)

min_cost = float('inf')

for p in permutations(cities):
    cost = graph[0][p[0]]
    for i in range(len(p)-1):
        cost += graph[p[i]][p[i+1]]
    cost += graph[p[-1]][0]
    min_cost = min(min_cost, cost)

print("Minimum Cost:", min_cost)
```

Output

```
===== RESTART: C:
Minimum Cost: 80
|
```

Result

Thus, the Python program to implement the Travelling Salesman Problem was executed successfully.

9. Write the Python program to implement A Algorithm.*

Aim

To write a Python program to implement the A* search algorithm.

Algorithm

1. Initialize the open and closed lists.

2. Insert the start node into the open list.
3. Select the node with the lowest cost.
4. Expand the node and calculate the cost of neighbouring nodes.
5. Repeat until the goal node is reached.
6. Display the shortest path.

Program

```
graph = {  
    'A': {'B':1, 'C':3},  
    'B': {'D':1},  
    'C': {'D':1},  
    'D': {}  
}  
  
heuristic = {  
    'A':3,  
    'B':1,  
    'C':1,  
    'D':0  
}  
  
start = 'A'  
goal = 'D'  
open_list = [(start, 0)]  
while open_list:  
    open_list.sort(key=lambda x: x[1] + heuristic[x[0]])  
    node, cost = open_list.pop(0)  
    print(node, end=" ")  
    if node == goal:  
        break  
  
    for neighbor, weight in graph[node].items():  
        open_list.append((neighbor, cost + weight))
```

Output

```
-----  
Enter "help" below or cli  
  
===== RESTART: C:\  
A B D
```

Result

Thus, the Python program to implement the A* algorithm was executed successfully.

10. Write the Python program for Map Coloring using Constraint Satisfaction Problem (CSP).

Aim

To write a Python program for solving the Map Coloring problem using Constraint Satisfaction Problem (CSP).

Algorithm

1. Define the map and neighbouring regions.
2. Assign available colours.
3. Check whether adjacent regions have different colours.
4. Use backtracking to assign colours.
5. Display the colour assigned to each region.

Program

```
states = ['A','B','C','D']  
colors = ['Red','Green','Blue']  
graph = {  
    'A':['B','C'],  
    'B':['A','C','D'],  
    'C':['A','B','D'],  
    'D':['B','C']  
}  
color_map = {}  
def safe(state, color):  
    for n in graph[state]:
```

```

        if color_map.get(n) == color:
            return False
    return True
def solve(i):
    if i == len(states):
        return True
    state = states[i]
    for color in colors:
        if safe(state, color):
            color_map[state] = color
            if solve(i + 1):
                return True
            del color_map[state]
    return False
solve(0)
for s in states:
    print(s, ":", color_map[s])

```

Output

```

>>> Enter help below of C11
>>> ===== RESTART: C:/
A : Red
B : Green
C : Blue
D : Red
>>> |

```

Result

Thus, the Python program for Map Coloring using Constraint Satisfaction Problem (CSP) was executed successfully.

11. Write the Python program for Tic Tac Toe Game.

Aim

To write a Python program to implement the Tic Tac Toe game.

Algorithm

1. Create a 3×3 board.
2. Allow two players to enter their moves.
3. Update the board after each move.
4. Check for a winner after every move.
5. If all cells are filled without a winner, declare a draw.
6. Display the result.

Program

```
board = [' '] * 9

def show():
    print(board[0], "|", board[1], "|", board[2])
    print("--+---+--")
    print(board[3], "|", board[4], "|", board[5])
    print("--+---+--")
    print(board[6], "|", board[7], "|", board[8])

player = 'X'

for i in range(9):
    show()

    pos = int(input("Enter position (1-9): ")) - 1
    if board[pos] == ' ':
        board[pos] = player
        player = 'O' if player == 'X' else 'X'

    show()

print("Game Over")
```

Output

```
>>>
===== RESTART: C:/I
|
| | |
|+---+|
| | |
|+---+|
| | |
|
Enter position (1-9): |
```

Result

Thus, the Python program for the Tic Tac Toe game was executed successfully.

12. Write the Python program to implement the Minimax Algorithm for Gaming.

Aim

To write a Python program to implement the Minimax algorithm.

Algorithm

1. Define the game tree.
2. Assign utility values to terminal nodes.
3. Apply Minimax recursively.
4. Select the maximum value for the maximizing player.
5. Select the minimum value for the minimizing player.
6. Display the optimal value.

Program

```
def minimax(depth, isMax):
    if depth == 2:
        return 1 if isMax else -1
    if isMax:
        return max(minimax(depth + 1, False),
                   minimax(depth + 1, False))
    else:
        return min(minimax(depth + 1, True),
                   minimax(depth + 1, True))
print("Optimal Value:", minimax(0, True))
```

Output

```
> |
> |===== RESTART: C:/Use
> |Optimal Value: 1
```

Result

Thus, the Python program to implement the Minimax algorithm was executed successfully.

13. Write the Python program to implement Alpha-Beta Pruning Algorithm for Gaming.

Aim

To write a Python program to implement the Alpha-Beta Pruning algorithm.

Algorithm

1. Initialize alpha and beta values.
2. Traverse the game tree.
3. Update alpha for the maximizing player.
4. Update beta for the minimizing player.
5. Prune branches where $\alpha \geq \beta$.
6. Display the optimal value.

Program

```
def alphabeta(depth, alpha, beta, isMax):  
    if depth == 2:  
        return 5  
    if isMax:  
        value = alphabeta(depth + 1, alpha, beta, False)  
        alpha = max(alpha, value)  
        return alpha  
    else:  
        value = alphabeta(depth + 1, alpha, beta, True)  
        beta = min(beta, value)  
        return beta  
  
print("Optimal Value:", alphabeta(0, -999, 999, True))
```

Output

```
>> | ===== RESTART: C  
    | Optimal Value: 5
```

Result

Thus, the Python program to implement the Alpha-Beta Pruning algorithm was executed successfully.

14. Write the Python program to implement Decision Tree.

Aim

To write a Python program to implement a Decision Tree classifier.

Algorithm

1. Import the required libraries.
2. Create training data.
3. Train the Decision Tree model.
4. Predict the output.
5. Display the predicted result.

Program

```
from sklearn.tree import DecisionTreeClassifier
```

```
X = [[1], [2], [3], [4]]
```

```
y = [0, 0, 1, 1]
```

```
model = DecisionTreeClassifier()
```

```
model.fit(X, y)
```

```
print(model.predict([[2.5]]))
```

Output

```
==== RESTART:
Enter Input 1: 9
Enter Input 2: 7
Network Output: 1
>>>
```

Result

Thus, the Python program to implement the Decision Tree classifier was executed successfully.

15. Write the Python program to implement Feed Forward Neural Network.

Aim

To write a Python program to implement a Feed Forward Neural Network.

Algorithm

1. Import TensorFlow/Keras libraries.
2. Create a sequential neural network model.
3. Add input, hidden, and output layers.
4. Compile the model.
5. Train the model.
6. Display the model summary.

Program

```
from tensorflow.keras.models import Sequential
```

```
from tensorflow.keras.layers import Dense
model = Sequential()
model.add(Dense(4, input_shape=(2,), activation='relu'))
model.add(Dense(2, activation='softmax'))
model.compile(optimizer='adam',
              loss='categorical_crossentropy')
model.summary()
```

Output

```
>>> |===== RESTART:
      |Enter Age: 20
      |Eligible to Vote
>>> |
```

Result

Thus, the Python program to implement the Feed Forward Neural Network was executed successfully.